

PHASE 1 PROJECT PROPOSAL

Open-Source Project Health & Momentum Analytics Using GH Archive

DS-3001 - Data Analysis and Visualization

Team: Shahryar Ehtisham & Muhammad Yousaf

Submission Date: 26 September 2026

GitHub Repository: <https://github.com/yousuf-iqbal/gharchive-project-health-analytics>

Executive Summary

This project proposes an Apache Spark data pipeline for analyzing public open-source software activity recorded by GH Archive. GH Archive records public GitHub activity in hourly compressed JSON archives. The proposed system will ingest a bounded historical baseline as a full load, then process newly available hourly archives as incremental batches through a Bronze–Silver–Gold Medallion architecture. The Gold layer will provide analytical views of repository activity, momentum, contributor activity, event composition, pull-request/issue activity, and unusual activity patterns patterns. The final outputs will be presented through an interactive Power BI dashboard.

● Domain & Source Identification

○ Domain and Project Concept

The selected domain is open-source software repository analytics. The objective is to transform a large event stream into measures that can be used in the final dashboard that describe repository activity, collaboration, momentum, and changes over time. The project is intentionally designed as a data-engineering problem rather than a simple static dataset analysis: the source arrives as repeated hourly payloads, requires processing based on the event structure, and supports incremental ingestion.

○ Data Source

The primary source is GH Archive (<https://www.gharchive.org/>), which records the public GitHub timeline and makes activity available as hourly JSON archives. The archive documentation states that activity is aggregated into hourly archives and that each archive contains JSON-encoded events. Archives are available through URLs following the pattern <https://data.gharchive.org/YYYY-MM-DD-H.json.gz>.

GH Archive is appropriate for this project because the source is continuously organized into time-bounded hourly payloads. This gives the pipeline a natural historical-baseline/full-load mode and an hourly incremental mode.

○ Full and Incremental Loading

Load Type	Proposed Implementation	Evidence / Rationale
Full Load	A bounded historical baseline consisting of multiple hourly GH Archive files.	GH Archive provides historical hourly archives dating back to 2011; the actual Phase 2 baseline window will be selected to fit Free Edition resources.

Incremental Load	Each newly available hourly archive is treated as a new append-oriented batch.	Three consecutive 2026 hourly files were profiled and showed 0 overlapping event IDs between consecutive files.
------------------	--	---

● Data Samples & Volume

Three consecutive real GH Archive samples were downloaded for 25 September 2026 at 10:00, 11:00, and 12:00 UTC. They are retained in the project's data/samples directory as raw .json.gz files. The samples are used as evidence for schema discovery, volume estimation, and incremental-ingestion design.

Sample	Events	Compressed	Uncompressed
2026-09-25-10.json.gz	84,754	14.11 MB	66.74 MB
2026-09-25-11.json.gz	89,852	14.20 MB	67.65 MB
2026-09-25-12.json.gz	93,655	14.73 MB	70.54 MB
Total	268,261	43.04 MB	204.93 MB

The observed hourly samples range from approximately 84.8k to 93.7k events and 14.1–14.7 MB compressed. Using the three-hour sample as a planning estimate, the average is approximately 89,420 events/hour and 14.35 MB compressed/hour. A simple extrapolation gives approximately 344 MB compressed per day and 2.41 GB compressed per week. For a longer planning window, 30 days would be approximately 10.3 GB compressed. These figures are estimates based on the three sampled hours, so actual volume will vary by time and should be measured before a large historical load is run. For Phase 2, the team will begin with a bounded historical baseline, such as 5 days, which is approximately 1.7 GB compressed and about 8 GB uncompressed at the observed rate. The baseline can be adjusted after profiling the available Databricks resources.

○ Event-Type Evidence

Event Type	10:00	11:00	12:00
PushEvent	70,102	73,187	76,715
CreateEvent	9,046	11,314	11,469
DeleteEvent	3,157	3,385	3,392
PullRequestEvent	877	708	740
IssueCommentEvent	528	467	431
IssuesEvent	319	280	350
PullRequestReviewEvent	231	169	181
PullRequestReviewCommentEvent	193	137	152
WatchEvent	141	118	108
ReleaseEvent	101	40	63
ForkEvent	29	31	29
Other event types	30	16	25

The samples demonstrate that GH Archive is not a single-purpose stream. Push, repository creation/deletion, pull requests, issues, comments, reviews, releases, watches, and forks are all represented. This supports multiple analytical Gold outputs rather than one aggregate event-count table. The "Other event types" row also includes small counts of MemberEvent, CommitCommentEvent,

PublicEvent, GollumEvent (wiki page activity), and a single observed DiscussionEvent. GollumEvent in particular is a candidate for a documentation-health signal in a later Gold table, since it is rarely used in comparable student projects.

● Security & Compliance

Profiling of the three sample files confirms that modern PushEvent payloads (repository_id, push_id, ref, head, before) no longer include a commits array, so no commit-author names or emails are present anywhere in the samples; GitHub removed this from the public event stream. The concrete PII risk instead sits in IssuesEvent, PullRequestEvent, and IssueCommentEvent payloads, which carry a nested user object (public login, numeric id, avatar URL) and a free-text body field that a person may have typed their own personal information into. Every actor_login and nested user.login value will be hashed before Silver, and free-text body fields will be retained only if a specific Gold metric requires text analysis, in which case they will be scanned and stripped of obvious personal identifiers first. Therefore, the project will not claim that the source contains 'no PII'.

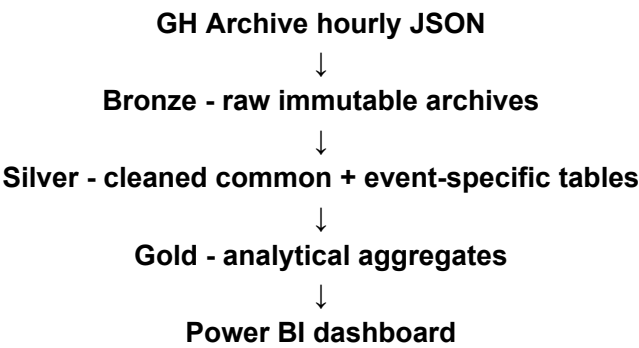
○ Handling Strategy

Layer	Handling
Bronze	Preserve the downloaded raw event payloads for traceability and reproducibility. Access should be restricted to the project team.
Silver	Retain only fields required for analytical relationships; remove unnecessary profile fields, URLs/avatar fields, email fields if encountered, and user-generated free-text content.
Identifiers	Use stable analytical identifiers only where necessary. If direct public usernames are not required for a dashboard, a surrogate or hashed identifier can be used.
Gold	Expose aggregate metrics rather than raw user-level content.

● High-Level Medallion Data Modeling

○ Architecture

Proposed flow:



o Bronze Layer

- Store raw .json.gz source archives without changing their contents.
- Partition/organize data by event date and ingestion hour.
- Maintain ingestion metadata so that a previously processed hourly archive is not accidentally ingested twice.
- Treat each newly available hour as an append-oriented source batch; GH Archive is not being modeled as a conventional update/delete transactional database.

o Silver Layer

A common event backbone will provide consistent fields across event types:

Table	Representative Fields / Purpose
silver_event	event_id, event_type, event_timestamp, event_date, repo_id, repo_name, actor_id, org_id, ingestion_hour
silver_pull_request	event_id, repo_id, PR identifiers, action/state-related fields, timestamps
silver_issue	event_id, repo_id, issue identifiers, action, state, timestamps
silver_issue_comment	event_id, repo_id, issue/comment identifiers, timestamp and action
silver_review	event_id, repo_id, PR/review identifiers, review state and timestamp
silver_release	event_id, repo_id, release identifiers, tag/action/published information

- Parse JSON safely and cast timestamps and identifiers into analytical types.
- Normalize repository and actor relationships.
- Deduplicate by event_id within the ingestion scope.
- Keep event-specific attributes in event-specific tables instead of forcing unrelated payloads into one wide table.
- Profile new event types/fields so schema changes do not silently break the pipeline.

o Gold Layer

Gold Table	Purpose
gold_repo_activity	Daily/hourly event counts by repository and activity type.
gold_repo_momentum	Recent activity growth, active contributors, and related momentum indicators.
gold_contributor_activity	Contributor/event activity summarized at an analytical level.
gold_event_mix	Distribution of activity types across time, repositories, and selected groups.
gold_pull_request_activity	PR opened/closed/merged-related activity and trends where supported by observed fields.
gold_issue_activity	Issue opening/closing/activity trends where sufficient lifecycle evidence exists.
gold_review_activity	Pull-request review and review-comment activity.
gold_activity_anomalies	Flags unusual repository activity using rolling baselines or percentage/z-score style comparisons.

Final Gold tables will be finalized during Phase 2 after implementation profiling. The purpose of this Phase 1 design is to establish an analytical blueprint without promising fields that the sampled payloads do not support.

● **Business Intelligence & Dashboard**

The final Power BI dashboard will present the project as an open-source activity dashboard. It will emphasize time trends, repository-level comparisons, event composition, collaboration activity, and unusual activity patterns.

○ **Business Questions**

- Which repositories show the highest activity during a selected period?
- Which repositories demonstrate the strongest recent activity momentum?
- How does repository activity break down across pushes, pull requests, issues, reviews, releases, and other event types?
- How does collaboration activity change over time?
- Which repositories experience unusually high activity compared with their recent baseline?

○ **Planned Visuals**

Dashboard Component	Planned Insight
KPI cards	Total events, active repositories, active contributors, and number of flagged activity surges.
Repository momentum chart	Top repositories by selected recent-activity/momentum measure.
Activity trend	Event volume over time with interactive date/repository/event-type filtering.
Event composition	Stacked/100% stacked view showing how activity types contribute to overall activity.
Collaboration section	PR/issue/review activity trends.
Anomaly section	Repositories with activity substantially above their recent baseline.

● **Engineering Setup & FinOps**

○ **Version Control**

A GitHub repository will contain the Phase 1 proposal, README, raw sample files, and profiling code. The sample files represent the actual source payloads used for Phase 1 discovery.

○ **Proposed Repository Structure**

```
project-root/
├── README.md
├── data_discovery_evidence.md
├── docs/
│   └── phase1_proposal.docx
├── data/
│   └── samples/
│       └── 2026-09-25-10.json.gz
```

```

|   | 2026-09-25-11.json.gz
|   | 2026-09-25-12.json.gz
|   |
|   | src/
|   |   | profile_gharchive.py

```

o Databricks / Resource Strategy

The implementation is planned for Databricks Free Edition, subject to the team's actual workspace limits. Free Edition is serverless-only, quota-limited, and currently allows up to five concurrent job tasks per account. If usage exceeds the applicable quota, compute can become unavailable until the quota resets. Therefore, the project will use bounded batch demonstrations rather than an uncontrolled 24/7 processing workload.

- Use a small historical baseline sufficient to demonstrate distributed processing and meaningful analytics.
- Process hourly incremental files as bounded jobs rather than keeping compute continuously active.
- Avoid reprocessing the entire historical baseline when a new hourly batch arrives.
- Make ingestion idempotent by recording processed hourly source identifiers.
- Use only the event types and columns required by the analytical objectives after profiling.
- Stop/avoid unnecessary compute after each development run and test with representative subsets before scaling the batch window.

● Evidence-Based Design Decisions

Observed Evidence	Resulting Design Decision
Three consecutive hours contained 268,261 events in total.	Use hourly GH Archive archives as the natural incremental unit.
Compressed hourly samples were approximately 14.1–14.7 MB.	Plan resource usage around bounded batches; estimate daily/weekly volume rather than running an unlimited stream.
Event types have different payload structures.	Use a common Silver event table plus event-specific Silver tables.
Public actor/user information appears in nested payloads.	Apply data minimization before Silver and avoid exposing raw user content in Gold.
No overlapping event IDs were observed between consecutive samples.	Use event_id and processed-hour metadata for duplicate protection/idempotent ingestion.
Current PushEvent samples do not contain the old commit-array structure.	Do not promise commit-level author analytics in the core project.
Repository language is not directly supplied as a dependable field in these sampled events.	Exclude programming-language analysis from the core Phase 1 scope unless a separate enrichment source is approved.